

Polyglot AI Team OS

可执行开发计划版：把路线图拆成明确模块、文件、阶段目标与验证点

这份版本不再只讲方向，而是把路线图继续压成工程执行语言：先做什么模块、改哪些文件、每阶段完成标准是什么、什么算做完。

它的核心目的只有一个：让接下来真正进入开发时，不再凭感觉扩功能，而是按节奏推进。

1. 总目标

把当前 Polyglot AI Team OS 从一个“已经有 runtime 雏形的原型仓库”，推进成一个真正可解释、可转向、可低成本运行、可持续迭代的终端版 **Team Runtime**。

这份计划不追求一口气做满，而追求三件事：

- 先拿回解释权：主 Agent 能看懂当前项目。
- 先做稳闭环：终端 → 计划 → 执行 → 测试 → 自愈 → 报告 → 交接。
- 再做 profile 化 Worker Fabric：在不爆复杂度的前提下引入多模型调度。

2. 当前代码骨架怎么理解

模块	当前作用	下一步角色
polyglot_cli.py	终端入口、命令路由	保持主入口；增加更强的 planner 查询体验
polyglot_ai/main_agent.py	主控制层、轻量状态回复、run/plan/route	升级为最小可用 planner，加入工作区感知层
polyglot_ai/runtime.py	session、state、events、timeline、handoff、lock	继续做状态中枢；承接安全、profile、审计能力
polyglot_ai/v1_worker.py	当前核心 worker 自愈循环	继续承担 coding task 闭环，但逐步与 profile/launcher 解耦
polyglot_ai/agents.py	本地 agent 发现与选择	从“发现 CLI”逐步扩成“worker profile / capability registry”
feishu_bridge.py / feishu_listener.py	IM / Feishu 入口	保持最小协同入口，先收稳 contract
skills/	pitfall skills + host integration skills	作为经验层继续沉淀，不先做大

3. 总体开发阶段

阶段	主题	目标
Phase 0	主 Agent 看见项目	建立工作区感知层，让 planner 不再“空调度”

Phase 1	收稳最小 Team OS 闭环	稳定计划、状态、执行、控制、报告、交接
Phase 2	建立极简版 Worker Fabric	统一执行容器 + model profiles + launcher + retry
Phase 3	安全、Feishu、技能沉淀增强	把长期差异化层做实，但不失控

4. Phase 0 : 主 Agent 工作区感知层

目标

让主 Agent 能回答：当前目录里有什么、核心入口是什么、这是个什么项目、最近状态是什么。

要做什么

- 新增 `workspace_snapshot`：目录树、关键文件、语言分布、最近文件。
- 新增 `workspace_summary`：把 `snapshot` 压成 `planner` 可消费的文本摘要。
- 让 `chat_reply()` 支持“看看当前目录 / 这个项目现在长什么样 / `repo` 里有什么”。
- 让 `plan_goal()` 在生成 Team Plan 前先读取最小 `workspace summary`。

建议修改文件

- `polyglot_ai/runtime.py`：增加工作区扫描和摘要方法，或新增独立 `workspace helper`。
- `polyglot_ai/main_agent.py`：接入 `snapshot / summary`，并扩展 `chat / plan` 分支。
- `polyglot_cli.py`：可新增诸如 `/workspace` 或 `/repo` 查询命令。

完成标准

- 主 Agent 能输出简洁的 `repo` 概览。
- 用户问“当前目录有什么”时，不再只返回 `release` 文件。
- Team Plan 不再只依赖用户一句话，而会带一份项目上下文摘要。

验证方式

- 在 CLI 里问三类问题：目录结构、关键模块、当前项目状态。
- 确认输出能准确提到 `polyglot_cli.py`、`polyglot_ai/`、`skills/`、`feishu_bridge.py` 等。

5. Phase 1 : 收稳最小 Team OS 闭环

目标

把“终端发起 → 计划 → 执行 → 测试 → 自愈 → 状态 → 报告 → 交接”这条主线做稳，作为整套系统的第一真实支点。

要做什么

1. 收稳 Team Plan 的输出结构，别再让它只是一个好看的对象。
2. 收稳 Task Board、`timeline`、`report`、`handoff` 的展示一致性。
3. 检查 `v1_worker.py` 的关键状态：`filling / testing / healing / success / failed / paused / stopped` 是否都可复现。
4. 把 `steering` 和 `control signal` 行为收稳，尤其是 `/pause`、`/resume`、`/stop`。

5. 至少确定一条“真实 coding task”测试基准任务，用来反复回归验证。

建议修改文件

- `polyglot_ai/main_agent.py` : plan / report / board / history 文本一致性。
- `polyglot_ai/runtime.py` : 事件、timeline、handoff、状态文件读写稳定性。
- `polyglot_ai/v1_worker.py` : 关键状态流转、自愈循环、control signal checkpoints。
- `monitor.py` / `status.py` / `steer.py` : 状态观察与控制体验。

完成标准

- 一条标准任务可以稳定从 plan 跑到 final report。
- 中途发送 steering / pause / resume / stop 时，系统行为可预测。
- handoff 真能帮助下一轮继续，而不只是生成一个文件。

验证方式

- 选 1 个简单 coding task 与 1 个中等 coding task 反复跑。
- 分别测试无 steering 与有 steering 两种路径。
- 检查最终 artifacts、report、task_packets、events 是否能自治。

6. Phase 2 : 建立极简版 Worker Fabric

目标

不要直接做“全异构 agent 平台”，而是先把 Worker Fabric 收敛成：统一执行容器 + **model profiles** + **worker launcher**。

要做什么

- 定义 `model_profiles` 数据结构：profile id、display name、base_url、model、capability tags、cost tier、timeout。
- 定义 `worker_launcher`：根据 profile 注入环境变量，拉起统一执行容器子进程。
- 先选一种执行容器作为第一版，例如 Claude Code CLI。
- 把“选哪个 worker”细化成“选哪个 profile + 哪个 shell”。
- 加入最小 fallback：profile A 失败 -> profile B 重试。
- 加入最小审计：task_id、profile_id、latency、status、token（如可获取）。

建议修改文件

- `polyglot_ai/agents.py`：从 agent discovery 延展到 profile registry / capability metadata。
- `polyglot_ai/worker_adapters.py`：拆出 launcher 层，别把所有逻辑塞在 adapter 里。
- `polyglot_ai/v1_worker.py`：从直接依赖当前选中的 backend，逐步改为依赖 launcher / profile。
- `polyglot_ai/runtime.py`：承接 profile audit 和运行元数据。

完成标准

- 同一执行容器可切换不同 profile 跑任务。
- 至少能用 2 个 profile 对同一任务做差异化尝试。
- 失败时能切换 profile 重试，而不是只能整局重跑。

验证方式

- 一条任务用 profile A 跑，一条用 profile B 跑，对比结果。
- 构造一个故意失败路径，验证 fallback 是否触发。
- 验证并发起 2 个子进程时不会互相污染环境。

7. Phase 3 : 安全、Feishu、技能沉淀增强

目标

在主线稳定之后，把最重要的长期差异化层补起来，但仍然保持克制。

安全层

- 建立 `credential_store` : 不存明文 secret。
- 建立 `redaction` : `stdout/stderr/log/artifact` 统一脱敏。
- 明确主 Agent 只看 `profile id / capability` , 不接触 `secret`。

Feishu 层

- 把 Feishu 收稳在“发任务 / 查状态 / 改方向 / 收结果”的最小可用路径。
- 把 `bridge / listener / session contract` 统一起来。
- 补足最基本的错误处理与权限边界。

技能层

- 继续维护 `pitfall skills` 的统一格式。
- 把 `host integration contract` 和 `bridge` 行为严格对齐。
- 逐步记录真正有复利价值的经验，而不是堆文档。

建议修改文件

- `skills/` 、 `skills/polyglot-team-os/`
- `feishu_bridge.py` 、 `feishu_listener.py`
- 新增安全相关模块，如 `polyglot_ai/credential_store.py` 、 `polyglot_ai/redaction.py`

8. 每阶段“不要做什么”也必须写清楚

阶段	不要做什么
Phase 0	不要急着让主 Agent 直接变成一个会读全仓的大模型黑盒；先做结构化 workspace summary。
Phase 1	不要一边收稳闭环，一边再加很多新的 runtime 功能。
Phase 2	不要同时深度适配 Claude/Codex/OpenClaw/OpenHands 一大堆 CLI。
Phase 3	不要把安全层和 Feishu 层做成比主线产品还重的工程项目。

9. 具体开发清单：最先动哪些点

1. 给 MainAgent 增加工作区感知接口。
2. 让 chat_reply() 能正确回答“当前目录 / 当前项目”类问题。
3. 把 Team Plan 与 workspace summary 接起来。
4. 选定一个标准任务，作为闭环回归样本。
5. 检查并收稳 v1_worker.py 状态流转和 control signal checkpoint。
6. 定义 profile 数据结构与最小 launcher 接口。
7. 把 current selected agent 逐步升级为 selected profile + selected shell。
8. 最后再把 secret 和 profile 启动流程收进 Runtime。

10. 验收口径：什么时候算“这一阶段真的完成了”

阶段	验收口径
Phase 0	主 Agent 可以正确描述 repo 结构和当前工作区。
Phase 1	至少一条真实任务能稳定经过 plan → run → heal → report → handoff。
Phase 2	至少两个 profile 可用于同一执行容器，并支持最小 fallback。
Phase 3	主 Agent 不碰 secret；Feishu 最小协同路径稳定；skills contract 和实际行为对齐。

11. 最后一句话：把路线图变成开发节奏，而不是继续堆想法

真正的关键不是“你还能想到多少很强的未来能力”，而是你能不能把现在这套系统按顺序收实：

1. 先让主 Agent 看见项目。
2. 先让 Team OS 闭环稳定跑通。
3. 再让 Worker Fabric profile 化。
4. 再把安全、Feishu、技能沉淀做深。

只要这个顺序不乱，你这个项目就会越来越像你自己的系统；一旦顺序乱了，它就会再次变成“功能很多但你解释不清”的东西。